Data Science and Engineering (2025) 10:40–53
https://doi.org/10.1007/s41019-024-00267-6

RESEARCH PAPER

Common Neighbor Completion with Information Entropy for Link Prediction in Social Networks

Zhengyun Zhou[1,2,3] · Guojia Wan[1,2,3] · Bo Du[1,2,3]

Received: 15 July 2024 / Revised: 16 September 2024 / Accepted: 9 October 2024 / Published online: 24 January 2025
© The Author(s) 2025

Abstract
Link prediction is essential for identifying hidden relationships within network data, with significant implications for fields such as social network analysis and bioinformatics. Traditional methods often overlook potential relationships among common neighbors, limiting their effectiveness in utilizing graph information fully. To address this, we introduce a novel approach, Common Neighbor Completion with Information Entropy (IECNC), which enhances model expressiveness by considering logical neighbor relationships. Our method integrates a dynamic node function with a Message Passing Neural Network (MPNN), focusing on first-order neighbors and employing set-based aggregation to improve missing link predictions. By combining the information entropy of probabilistic predictions of common neighbors with MPNN and leveraging information entropy to assess uncertainty in adjacent connections, our approach significantly enhances prediction accuracy. Experimental results demonstrate that our IECNC method achieves optimal performance across multiple datasets, surpassing existing techniques. Furthermore, visualizations confirm that our model effectively captures and accurately learns feature information from various categories, Demonstrating the method's efficacy and adaptability.

Keywords Information entropy · Link prediction · Common neighbor · Social network

1 Introduction

Link prediction is an essential computational technique within the domain of network science, aimed at uncovering potential relationships within a network [1]. It plays a key role in understanding and predicting the evolution of complex networks in fields like social media [2, 3], biology [4, 5], and information systems [6, 7]. By predicting whether a specific link will occur between entities within a network [8], researchers can gain insights into the underlying structure and dynamics of the network [9], enhance recommender systems [7, 10, 11], and predict social interactions [12]. Link prediction helps to reveal hidden connections, improve user experience, and strengthen the overall connectivity of the network [13–15]. The task involves the use of various predictive models ranging from simple probabilistic algorithms to sophisticated machine learning methods, including graph neural networks [16, 17]. The ongoing advancements in this area continue to contribute to our understanding of how networks evolve and how information flows within them [18], making link prediction a fundamental aspect of modern network analysis [12]. Identifying and evaluating the relationships between pairs of nodes can enhance the precision of link prediction. This ability is crucial for improving the performance of predictive models, such as MPNN [19]. In MPNNs, nodes communicate by exchanging messages through layers, where each message is a function of the nodes' attributes and their connecting edges. This process is essential for updating node representations based on local neighborhood information. Various methods focus on capturing pairwise representations, which are at the core of advanced link prediction strategies. Notable among these are Graph Autoencoders (GAE) [20], the

✉ Guojia Wan
guojiawan@whu.edu.cn

✉ Bo Du
dubo@whu.edu.cn

[1] National Engineering Research Center for Multimedia Software, Institute of Artificial Intelligence, and School of Computer Science, Wuhan University, Wuhan 430072, Hubei, China

[2] Hubei Luojia Laboratory, Wuhan 430072, Hubei, China

[3] Hubei Key Laboratory of Multimedia and Network Communication Engineering, Wuhan University, Wuhan 430072, Hubei, China

Subgraph Embedding-based Link Prediction model (SEAL) [21], NEO-GNN [22], and BUDDY [23]. NCNC [24] model enhances link prediction by using learnable pairwise representations and addressing the problem of unobserved links. Each of these methods provides unique mechanisms for embedding node pairs into the graph's latent space, enhancing the model's ability to identify and reconstruct the complex web of node relationships. For example, GAEs encode and decode graph structures into a latent space, predicting links by reconstructing the adjacency matrix. SEAL extracts and classifies local subgraph structures around potential links, effectively using local connectivity patterns for prediction. NEO-GNN and BUDDY extend these concepts by incorporating novel architectural features and learning strategies, ensuring that the embeddings capture both the explicit and implicit relationships essential for high-accuracy link prediction.

However, in traditional link prediction methods, while some approaches focus on common neighbors and others explore higher-order relationships, they often fall short in fully leveraging the logical information provided by common neighbors. This limitation results in restricted expressiveness and less accurate predictions. Our method addresses this gap by explicitly incorporating the logical relationships among neighbors into the prediction model. By doing so, we enhance the model's ability to capture and represent the nuanced interactions between nodes, thus providing a richer and more comprehensive expressiveness that improves the accuracy of link prediction.

Our method, Common Neighbor Completion with Information Entropy (IECNC), synthesizes node representations by integrating shared common neighbors, thus avoiding reliance on handcrafted features. This common neighbor-based aggregation method renders node representations more flexible and learnable. To mitigate the impact of graph incompleteness on link prediction tasks, we utilize information entropy to quantify the uncertainty of each link. Specifically, we calculate the information entropy of the common neighbor probabilities and integrate it with a MPNN to update and optimize node representations. The use of information entropy effectively captures and leverages potential connections between nodes, particularly when there are unobserved links in the data. This approach not only fills in missing information in the graph but also significantly enhances the model's ability to handle link prediction in complex networks. The integration of information entropy has led to notable performance improvements across multiple datasets.As shown in Fig. 1, it shows a schematic of our model. First, we extract the logical relationships of neighbors from the subgraph and compute node features. Then, we use the information entropy neighbor completion method to obtain link predictions between any pair of nodes.

Empirical evaluations demonstrate that our method not only maintains scalability but also significantly outperforms existing models across various datasets. Our method achieved leading HITS@100 scores on the Cora, Citeseer, Pubmed, twitch_RU, and twitch_PT datasets, highlighting its effectiveness in diverse network environments. The integration of information entropy, which quantifies the uncertainty of connections through the entropy of common neighbor probabilities, significantly enhances the model's ability to capture subtle and often overlooked structural nuances in network data. This approach allows for more precise and robust link predictions by effectively utilizing and learning from the available structural information in networks. These results underscore the practical applicability and potential of our approach in accurately learning and utilizing network structural information, thereby providing more precise link predictions.

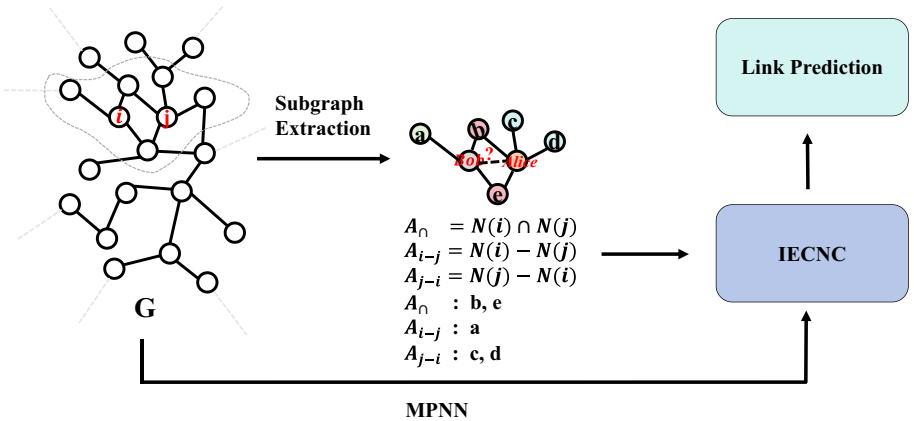


Fig. 1 Illustration of model. We extract the logical relationships of neighbors from the subgraph and compute node features. Then, we use the information entropy neighbor completion method to obtain link predictions between any pair of nodes

2 Related Works

2.1 Link Prediction

Link prediction heuristics [25] serve as fundamental tools in network analysis, aiming to predict the likelihood of a future or missing link between nodes in a network [26, 27]. Early techniques often employed simple proximity-based metrics, such as the Common Neighbors (CN) [28, 29]. Jaccard's coefficient and the Adamic-Adar index further refine this approach by normalizing and weighting the shared connections, enhancing the predictive accuracy in various contexts. NCNC [24] enhances link prediction by addressing graph incompleteness, employing a unique architecture that combines structural features with neural message passing to complete missing connections and improve prediction accuracy.

The integration of Graph Neural Networks (GNNs) [30] into link prediction tasks represents a significant advancement, leveraging the intricate dependencies inherent within graph data. GAT aggregate node feature representations by dynamically assigning attention weights among different neighboring nodes, with these weights reflecting the importance of the relationships between nodes. In link prediction tasks, GAT utilizes this mechanism to learn and predict potential connections between nodes from the graph, thereby enhancing the accuracy and effectiveness of link prediction. GNNs utilize deep learning techniques to process graphs directly, adapting the neural network architectures to respect the graph structure, thereby producing more informative, context-aware embeddings [31–33]. Subgraph Embedding-based Link Prediction (SEAL) [21]: SEAL approaches link prediction by extracting and labeling the subgraph surrounding a candidate link, and then treating link prediction as a graph classification task. This method is based on the hypothesis that the local subgraph structure around a potential link contains crucial information about the existence of that link. SEAL uses a labeling trick to enhance the subgraph's nodes with their roles (e.g., target nodes, neighbors), which improves the learning process.This embedding is then used to classify whether a link should exist between u and v. Graph Autoencoders (GAEs) [20]: GAEs are an unsupervised approach that learns to encode node features into a latent space and then reconstructs the graph's adjacency matrix from these embeddings.

MPNN [19]: MPNNs are particularly notable for their flexibility in defining how messages are passed between nodes, which is critical for effective learning. The general form of MPNN operations involves two functions across multiple iterations or layers.

In this framework, GCN [34] (Graph Convolutional Network) updates node features based on their neighbors, and σ is typically a logistic sigmoid function that converts the dot product of latent vectors into probabilities. These probabilities indicate the likelihood of links existing between nodes, making GAEs particularly effective for link prediction in graphs with missing edges.

2.2 Graph Data Incompleteness

Graph data incompleteness, characterized by missing nodes and edges, significantly complicates network analysis, leading to biased or inaccurate predictions. This problem arises because real-world networks are often incomplete, with many connections and nodes not fully observed. To address this challenge, researchers employ various techniques, including probabilistic models and matrix factorization methods, based on the assumption that the observed network is merely a fragment of a larger, unseen graph [35].

Probabilistic models [35] estimate missing links by calculating the likelihood of connections between nodes. These models evaluate the probability of a link between nodes u and v, using observed data to predict unobserved connections. This probabilistic approach helps in inferring missing edges and improving overall network accuracy.

Matrix factorization techniques complement probabilistic models by decomposing the graph's adjacency matrix into lower-dimensional latent factor matrices. The assumption here is that the graph's structure can be effectively represented by these latent factors. This approach not only reconstructs missing elements but also reveals underlying structures such as communities or influential nodes, which might not be apparent from incomplete data alone.

Given the complexity and variability of graph data, these advanced techniques, supported by solid statistical foundations, offer significant potential. They not only help in filling gaps in the graph data but also enhance our understanding of complex networks, leading to more accurate predictions and insights [36].

3 Preliminaries

3.1 Notations

We use a classical representation for undirected graphs, $G = (V, E, M, F)$: V is the node set. $E \subseteq V \times V$ is the set of edges between nodes. $F \in \mathbb{R}^{n \times d}$ is the node feature matrix, where each row represents the attributes of node v. M is the $n \times n$ adjacency matrix, symmetric, with $M_{uv} = w$, if $(u, v) \in E$, otherwise $M_{uv} = 0$. Here, w is typically 1 or a specific weight.

$N(u, M)$ is the neighbors of node u, $N(u, M) := \{v \mid M_{uv} > 0\}$. For a known and unchanged M, we use $N(u)$. The common neighbors of nodes i and j

$:N(i) \cap N(j)$. High-order adjacency matrix M^l represents paths of length l. The neighborhood set $N(u, M^l)$ is:$N(u, M^l) = \{v \mid (M^l)_{uv} > 0\}$. Nodes at an exact shortest path distance l from u in M are $N_l(u, M)$. High-order neighborhoods: $N(u, M^l)$: nodes connected by paths of length exactly l. $N_l(u, M)$: nodes at shortest path distance l from u. For u, the broad neighborhood might be represented as $N_{l_1}(u, M^{l_2})$, simplified to: $N^{l_1}_{l_2}(u) = N_{l_1}(u, M^{l_2})$, assuming a fixed adjacency matrix. For link analysis between nodes i and j, relevant overlaps and differences in their neighborhoods are: $Overlap : N^{l_1}_{l_2}(i) \cap N^{l_4}_{l_3}(j)$, $Difference : N^{l_1}_{l_2}(i) \backslash N^{l_4}_{l_3}(j)$.

3.2 Message Passing Neural Network

In the architecture of MPNN, each node v undergoes a series of transformations through multiple layers to aggregate and process information from its neighbors, refining the node's representation. The initial representation $h_j^{(0)}$ is set to the node's features. At each layer k, the node representation is updated by aggregating messages from its neighbors using an aggregation function (e.g., sum or max) and processing this aggregated information along with the node's current representation using functions typically realized as multi-layer perceptrons (MLPs). This process can be formally expressed as:

$$h_j^{(k+1)} = U^{(k)}\left(h_j^{(k)}, \mathrm{AGG}_{u \in N(j)}\left(L^{(k)}\left(h_i^{(k)}, h_j^{(k)}, e_{ij}\right)\right)\right) \tag{1}$$

where $h_j^{(k)}$ is the representation of node j at the kth layer, where $U^{(k)}$ and $L^{(k)}$ are functions such as a multi-layer perceptron, and AGG denotes an aggregation function like sum, which adds up all neighbor node features; max, which selects the maximum value among neighbor node features; mean, which averages the features of neighboring nodes; and attention, which dynamically weights the importance of each neighbor based on its features. After k layers, the node representation $h_j^{(k)}$ encapsulates not only the initial node features but also the structural and contextual information derived from its neighborhood. The final representation used as the output of the MPNN for node j is:

$$\mathrm{MPNN}(v, M, F) = h_j^{(K)} \tag{2}$$

3.3 Information Entropy

In information theory and probability statistics, entropy is a measure of the uncertainty of a random variable. Let X be a discrete random variable that takes a finite number of values, with a probability distribution $P(X = x_i) = p_i, i = 1, 2, \ldots, n$, then the entropy $H(X)$ of the random variable X is defined as: $H(X) = -\sum_{i=1}^{n} p_i \log p_i$, if $p_i = 0$, then it is defined that $0 \log 0 = 0$. Usually, the base of the logarithm is 2, the unit is bits, or the natural logarithm base e and the unit is nat. By definition, entropy depends only on the distribution p of X, and not on the values of X, hence X's entropy is denoted as $H(p)$, i.e.:

$$H(p) = -\sum_{i=1}^{n} p_i \log p_i \tag{3}$$

3.4 Link Prediction

In the context of MPNNs, the node representations are updated in each layer by aggregating information from neighboring nodes. After k layers of message passing, the final node representations are used to predict the existence of links between nodes u and v.

A specific MPNN model for link prediction can be defined using the following formula:

$$\hat{y}_{ij} = \sigma(h_i, h_j, C_{ij}), \tag{4}$$

where σ is a non-linear activation function, h_i and h_j are the node representations of i and j, and C_{ij} represents the interaction between nodes i and j.

4 Method

4.1 Modeling Logical Context on Node Pair

In the study of social networks, understanding the nuanced relationships between users is crucial for accurate modeling. We define several logical predicates to capture different aspects of user interactions:

q_i : $u \to i$, User u is following User i
q_j : $u \to j$, User u is following User j
$q_i \wedge q_j$: User u is following both User i and User j
$q_i \vee q_j$: User u is following either User i or User j
$q_i - q_j$: User u is following User i, but not following User j
$q_j - q_i$: User u is following User j, but not following User i

As depicted in Fig. 2, the knowledge graph G represents a social network where nodes denote users and edges denote their follow relationships. Here, Bob and Alice correspond to nodes i and j respectively. Node a, b, e represents users who follow Bob, denoted as q_i, while nodes b, c, d, e represent users who follow Alice, denoted as q_j. Node b, e represents users who follow both Bob and Alice, denoted as $q_i \wedge q_j$. Nodes a, b, c, d, e collectively represent users who follow either Bob or Alice, denoted as $q_i \vee q_j$. Node a denotes users who follow Bob but not Alice, denoted as $q_i - q_j$, while nodes c and d denote users who follow Alice but not Bob, denoted as $q_j - q_i$. These common neighbors, linked through logical operations such as difference, intersection,

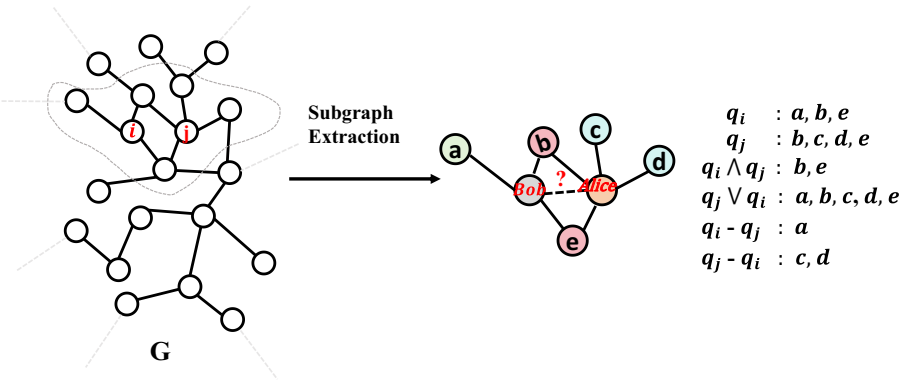


Fig. 2 Illustration of modeling Logical Context on Node Pair. In a social network, to predict if there is a following relation between Bob and Alice, we can consider the logical relationships of their common neighbors to achieve more accurate link predictions

and union, model the logical neighbor relationships, in turn, constructs the logical context for relationships.

These predicates collectively form the logical context C_{ij} for a node pair (i, j):

$$C_{ij} = \{q_i, q_j, q_i \wedge q_j, q_i \vee q_j, q_i - q_j, q_j - q_i\} \tag{5}$$

Here, C_{ij} encapsulates the various logical relationships between users i and j. This logical structure serves as the foundation for integrating more detailed embeddings and representations into our model.

Following this logical foundation, we integrate these concepts into our model architecture:

$$\begin{aligned}
\mathbf{C}_{ij} &= [\mathbf{h}_i; \mathbf{h}_j; \mathbf{Q}_\vee; \mathbf{Q}_\wedge; \mathbf{Q}_{i-j}; \mathbf{Q}_{j-i}], \\
\mathbf{Q}_\vee &= MLP([\mathbf{h}_0; \cdots; \mathbf{h}_k; \cdots), h_k \in q_i \vee q_j, \\
\mathbf{Q}_\wedge &= MLP([\mathbf{h}_0; \cdots; \mathbf{h}_k; \cdots), h_k \in q_i \wedge q_j, \\
\mathbf{Q}_{i-j} &= MLP([\mathbf{h}_0; \cdots; \mathbf{h}_k; \cdots), h_k \in q_i - q_j, \\
\mathbf{Q}_{j-i} &= MLP([\mathbf{h}_0; \cdots; \mathbf{h}_k; \cdots), h_k \in q_j - q_i.
\end{aligned} \tag{6}$$

This logical approach is integrated into our model, where $\mathbf{C}_{ij}$ for a node pair (i, j) includes feature representation $\mathbf{h}_i$ and $\mathbf{h}_j$, as well as logical representations $\mathbf{Q}_\vee$, $\mathbf{Q}_\wedge$, $\mathbf{Q}_{i-j}$, and $\mathbf{Q}_{j-i}$. These logical representations are computed using MLPs that process the logical relationships $q_i \vee q_j$, $q_i \wedge q_j$, $q_i - q_j$, and $q_j - q_i$, providing a comprehensive representation of the node pair's interaction dynamics. This holistic approach enhances our model's ability to capture the nuanced relationships within the social network, integrating both structural and logical information to predict and understand node interactions more effectively.

To predict the likelihood of a connection between nodes u and v, we utilize a neural network model where $\hat{y}_{uv}$ is computed as:

$$\hat{y}_{ij} = \sigma(h_i, h_j, C_{ij}) \tag{7}$$

where h_u and h_v represent the embeddings or features of nodes u and v, respectively, and σ is an activation function (e.g., sigmoid) mapping the combined features and logical context C_{uv} to a predicted probability.

This approach enables the integration of logical context into the modeling of node interactions, enhancing the model's ability to capture complex relationship dynamics in social networks.

4.2 Learning Logical Context Representation

Guided by the results of our preliminary analysis, we develop a strategy to amend the social network. Initially, we employ a link prediction model to infuse the graph with inferred links, thus partially rectifying its incompleteness. Subsequently, the enhanced graph undergoes a second application of the model to finalize the link predictions. This dual-step process ensures that the model utilizes the additional information from inferred links, leading to more accurate and reliable link predictions.

Firstly, we define different sets of neighbors: $A_\cap = q_i \wedge q_j$ denotes the common neighbors of nodes i and j, $A_{i-j} = q_i - q_j$ denotes neighbors of node i not connected to node j, and $A_{j-i} = q_j - q_i$ denotes neighbors of node j not connected to node i.

Next, we introduce the Common Neighbor Encoder, in our model, the probability P_{uij} that a node u influences the link between nodes i and j is determined as follows:

$$P_{uij} = \begin{cases} 1, & \text{if } u \in A_\cap, \\ \text{softmax}(\mathbf{Q}_{i-j}), & \text{if } u \in A_{i-j}, \\ \text{softmax}(\mathbf{Q}_{j-i}), & \text{if } u \in A_{j-i}, \\ 0, & \text{otherwise.} \end{cases}$$

Here, $\mathbf{Q}_{i-j}$ and $\mathbf{Q}_{j-i}$ are computed using MLPs and represent the interaction strengths of nodes i and j with their respective neighbors. This approach ensures that the model can capture both explicit and implicit relationships between nodes, leading to more accurate and expressive link predictions. The illustration is shown in Fig. 2. The following conditions are posited: if both edges (i, u) and (j, u) are observable within the graph, it is established that u is a common

neighbor of i and j. Conversely, should one of the edges (i, u) or (j, u) remain undetected, the model is deployed to forecast the probability of its existence. This probability also serves as the estimate for u's likelihood of being a common neighbor. In the scenario where both edges (i, u) and (j, u) are absent, the probability of common adjacency is deemed negligible and thus set to zero.

The application of softmax transforms the outputs into a distribution over potential categories or relationships, providing a probabilistic interpretation of the outputs. This transformation aids in a more intuitive understanding of the model's predictions, allowing practitioners to interpret link prediction outcomes in a meaningful and actionable way. Additionally, softmax significantly enhances the stability of gradient descent during the training phase, promoting faster convergence and more efficient optimization of model parameters. Moreover, by normalizing the outputs and accentuating the differences in predictions, the softmax function introduces a competitive mechanism among the output classes. This feature is particularly beneficial in link prediction tasks, where accurately distinguishing between different types of relationships or probabilities of links is essential. Due to the varying gradient propagation efficiency of different activation functions, softmax is well-suited for the backpropagation process, thereby further improving the training efficiency of the model. In summary, the integration of softmax activation into the computation of P_{uij} not only boosts the model's performance but also establishes a robust, interpretable, and stable framework for link prediction across intricate network structures.

Traditional models predominantly rely on static or degree-based node functions in modeling node relationships, which often fall short in capturing complex structural and attribute information spanning multiple hops. To address these limitations, we propose the integration of a GNN as a dynamic node function, enhancing the extraction of pairwise features as described below:

$$S(q) = \bigoplus_{v \in q} MPNN(v, M, F) \tag{8}$$

Here, $\oplus$ denotes a set-based aggregation operation, tailored to combine features effectively from first-order neighbors. MPNN serves as a versatile feature extractor, capable of adapting to various node characteristics, significantly outperforming traditional functions that only consider node degrees.

The choice of employing MPNNs over other graph neural network architectures is driven by several key factors that make MPNNs particularly adept for handling complex graph structures. Firstly, MPNNs offer a flexible messaging mechanism that allows for the customization of message functions and aggregation processes tailored to specific graph characteristics. This adaptability enables MPNNs to effectively capture complex dependencies among nodes, incorporating features from direct and higher-order neighbors through iterative message passing. Moreover, MPNNs are highly suitable for graphs with intricate connectivity patterns and rich edge attributes, such as molecular structures, where edge features play a crucial role. Unlike some graph neural networks that simplify edge features, MPNNs can directly process and leverage these attributes, enhancing the model's ability to learn detailed interactions between nodes.

Upon integrating the MPNN, initial investigations suggested that higher-order neighbor features can be implicitly represented by first-order neighbors. This observation led us to hypothesize that MPNNs might inherently learn higher-order dependencies within their architecture. To optimize for scalability and efficiency, we thus streamlined the model to focus exclusively on first-order neighbors, using only intersection operations for set aggregation.

To address the issue of graph incompleteness, we introduce an intervention method called Common Neighbor Completion (CNC). This method mitigates the impact of graph incompleteness on link prediction accuracy by predicting potential common neighbors to complete the graph structure.

Building on this, we propose a new approach called Probabilistic Prediction Neighbor Completion (PPNC). The PPNC method integrates the aforementioned probability P_{uij} into the link prediction model. By combining the characteristics of MPNN and using probabilistic completion of common neighbor relationships, PPNC further enhances the accuracy and robustness of link prediction. Thus, our model not only addresses graph incompleteness but also captures complex node relationships, providing more accurate link predictions.

$$\text{PPNC}(i, j) = \sum_{u \in q} P_{uij} \text{MPNN}(u, M, \mathbf{C}_{ij}) \tag{9}$$

where $\text{PPNC}(i, j)$ represents the predicted link score between nodes i and j. The summation is performed over all common neighbors u in the set q. For each common neighbor u, the probability P_{uij} quantifies the likelihood that u plays a significant role in linking i and j. $\text{MPNN}(u, M, \mathbf{C}_{ij})$ denotes the output of the Message Passing Neural Network (MPNN) for node u, considering context information $\mathbf{C}_{ij}$. By combining the MPNN output with the probability P_{uij}, this formula captures the potential complex relationships between i and j, leading to more accurate link prediction results.

4.3 Common Neighbor Completion with Information Entropy

Inspired by information entropy, we introduce a novel approach to measure the uncertainty of predicted

probabilities. We use the information measure $-p \log p$ to represent the quantified uncertainty of node v being a common neighbor of nodes i and j. This measure captures the amount of uncertainty associated with the prediction. If the probability p is close to 0.5, it indicates maximum uncertainty, as the event is equally likely to occur or not occur. Conversely, probabilities close to 0 or 1 indicate low uncertainty.

Similarly, the total uncertainty for all nodes u being common neighbors of endpoints i and j can be defined as:

$$-\sum_{u \in q} P_{uij} \ln P_{uij} \tag{10}$$

This formula sums up the individual uncertainties across all nodes u regarding their potential as common neighbors of i and j. A higher value indicates greater dispersion in probabilities, reflecting higher uncertainty in the model's predictions. This approach evaluates the uncertainty of the connection between i and j by considering all possible common neighbors. The more dispersed the probability distribution (i.e., the probabilities of each node becoming a common neighbor are close and not concentrated), the higher the information entropy, indicating greater uncertainty. Combining this uncertainty assessment results in our new model:

$$\text{IECNC}(i,j) = -\sum_{u \in q} P_{uij} \ln P_{uij} \cdot \text{MPNN}(u, M, \mathbf{C}_{ij}) \tag{11}$$

In this equation, $\text{IECNC}(i,j)$ represents the score for the link prediction between nodes i and j, incorporating the information entropy of their common neighbors. The summation is performed over all common neighbors u in the set q. Each term $-P_{uij} \ln P_{uij}$ reflects the information content contributed by node u in reducing the uncertainty of predicting the link between i and j. This term is based on the concept of information entropy, where P_{uij} represents the probability of node u being a common neighbor of nodes i and j.

Furthermore, $\text{MPNN}(u, M, \mathbf{C}_{ij})$ denotes the Multi-Perceptron Neural Network function, which processes the features M and the context information $\mathbf{C}_{ij}$ of node u. The information term $-P_{uij} \ln P_{uij}$ signifies how much node u contributes to reducing the uncertainty in predicting the relationship between i and j. Nodes with P_{uij} values closer to 0 or 1 provide less information due to lower predictive uncertainty, while nodes with P_{uij} near 0.5 contribute more informative content by offering greater uncertainty reduction.

The goal of the model is to minimize this uncertainty, thus reducing the information entropy. By encouraging the model to assign probabilities P_{uij} that align more closely with actual occurrences, the predicted probabilities for nodes will tend more towards 1 or 0. This reduction in entropy indicates more confident and accurate predictions.

This approach, by integrating information entropy into the prediction process, not only provides a more nuanced evaluation of predictive uncertainty but also supports the model in achieving more accurate and reliable link predictions. The methodology ensures that the model can effectively distinguish between nodes that are likely and unlikely to be common neighbors, thereby enhancing the overall robustness and performance of the link prediction task.

5 Experiment

5.1 Dataset

Link prediction datasets are essential for training algorithms that predict unobserved edges in a graph, useful in various fields like social network analysis, bioinformatics, and recommender systems. In link prediction research, we selected four datasets: Cora, Citeseer, PubMed, and Twitch, as they encompass a range of network types from academic citation networks to social media networks. This selection not only demonstrates the generalizability of our method across diverse structures and dynamic networks but also broadens the applicability of our findings across various domains. Here is a concise overview of several types of datasets:

- **Social Networks**: *Facebook* (individuals and friendships) and *Twitter* (users and followings).
- **Collaboration Networks**: *DBLP* (authors and co-authorships), *ArXiv Astrophysics* (scientists and co-authorships).
- **Biological Networks**: *Protein-Protein Interactions (PPI)*, *Gene Regulatory Networks (GRN)* (genes and regulatory interactions).
- **Citation Networks**: *CiteSeer*, *PubMed Diabetes* (academic papers and citations).
- **Infrastructure Networks**: Examples include power grids with substations or power sites linked by transmission lines.
- **Recommender Systems**: *MovieLens* (users, movies, ratings/viewing patterns).

These datasets often deal with challenges like sparsity, evolving structures, and class imbalance, necessitating sophisticated link prediction methods.

Twitch: The Twitch Gamer networks introduced in the "Multi-scale Attributed Node Embedding" paper. Nodes represent gamers on Twitch and edges are followerships between them. Node features represent embeddings of games played by Twitch users. The task is to predict whether a user streams mature content.The Twitch dataset is publicly available at https://snap.stanford.edu/data/.

Cora: The Cora dataset is a widely-used benchmark in graph machine learning, consisting of 2,708 scientific publications each represented as a node within a citation network. Nodes are connected by edges representing citations, and each document is associated with a word vector derived from its abstract. The dataset includes seven classes of documents, corresponding to different areas of machine learning research, making it ideal for tasks such as node classification and link prediction in graph neural networks. The Cora dataset is publicly available at https://github.com/kimiyoung/planetoid.

Citeseer: The Citeseer dataset is a collection used frequently in graph learning applications, particularly in document classification tasks. It consists of 3,312 scientific publications categorized into six classes. Each document is represented as a node in a citation network, with edges denoting citations between papers. The dataset features a unique attribute-based representation of documents, using a binary word vector derived from the text.The Citeseer dataset is publicly available at https://github.com/kimiyoung/planetoid.

PubMed: The PubMed dataset comprises 19,717 scientific publications from the PubMed database related to diabetes. Classified into one of three categories (Diabetes Mellitus Experimental, Diabetes Mellitus Type 1, and Diabetes Mellitus Type 2), each publication is represented as a node in a citation network. Nodes are linked based on citations, and node features correspond to TF-IDF weighted word vectors from abstracts.The PubMed dataset is publicly available at https://github.com/kimiyoung/planetoid.

In the experimental section of this paper, numerous social network datasets relevant to link prediction are reviewed to illustrate the variety and scope of available data for this research. From these datasets, a select few have been chosen for detailed experimental analysis. The selection criteria were based on the diversity of network structures, and the availability of comprehensive link data, ensuring that our methods are both reliable and generalizable across different types of social networks.

5.2 Quantitative Experiments

In this study, we compared the performance of various link prediction methods across multiple datasets. As illustrated in Table 1, we evaluated a range of methods including traditional link prediction metrics (CN, AA, RA) as well as advanced graph neural network models (GCN, SAGE, SEAL, NBFnet, Neo-GNN, BUDDY), in addition to introducing our novel approach (denoted as IECNC).

The results clearly show that our method outperforms all other methods tested on each dataset. For instance, on the Cora dataset, our approach achieved a HITS@100 score of $94.09\% \pm 1.91\%$, significantly higher than the comparative methods. Similarly, on the Citeseer dataset, our method led with $95.34\% \pm 0.87\%$. Furthermore, for the Pubmed, *twitch_RU*, and *twitch_PT* datasets, our method reached scores of $85.72\% \pm 1.51\%$, $62.18\% \pm 4.4\%$, and $56.27\% \pm 1.34\%$ in HITS@100 respectively, further demonstrating its effectiveness across different types of network data.

Table 1 Performance metrics for various link prediction methods across datasets

Method Metric	Cora HITS@100	Citeseer HITS@100	Pubmed HITS@100	twitch_RU HITS@100	twitch_PT HITS@100
CN	33.92 ± 0.46	29.79 ± 0.90	23.13 ± 0.15	17.23 ± 0.00	16.15 ± 0.00
AA	39.85 ± 1.34	35.19 ± 1.33	27.38 ± 0.11	20.31 ± 0.00	19.53 ± 0.00
RA	41.07 ± 0.48	33.56 ± 0.17	27.03 ± 0.35	20.01 ± 0.00	19.33 ± 0.00
GCN	66.79 ± 1.65	67.08 ± 2.94	53.02 ± 1.39	37.75 ± 1.07	35.67 ± 1.32
SAGE	55.02 ± 4.03	57.01 ± 3.74	39.66 ± 0.72	31.10 ± 0.81	29.55 ± 2.40
SEAL	81.71 ± 1.30	83.89 ± 2.15	75.54 ± 1.32	50.74 ± 0.43	46.80 ± 3.16
NBFnet	71.65 ± 2.27	74.07 ± 1.75	58.73 ± 1.99	42.23 ± 1.12	33.53 ± 1.71
Neo-GNN	80.42 ± 1.31	84.67 ± 2.16	73.93 ± 1.19	50.52 ± 0.37	44.13 ± 0.60
BUDDY	88.00 ± 0.44	92.93 ± 0.27	74.10 ± 0.78	55.94 ± 0.58	47.85 ± 0.20
NCN	89.05 ± 0.96	91.56 ± 1.43	79.05 ± 1.16	57.76 ± 0.87	51.19 ± 0.85
NCNC	89.65 ± 1.36	93.47 ± 0.95	81.29 ± 0.95	58.61 ± 0.71	52.42 ± 0.73
IECNC	**94.68 ± 1.43**	**96.44 ± 1.63**	**87.56 ± 1.51**	**62.18 ± 2.4**	**56.27 ± 1.34**

Bold values indicate the best performance in the experimental results

Performance comparison of various link prediction methods across different datasets, evaluated using HITS@100 metric. The methods include common neighbor-based methods (CN [31], AA [33], RA [32]), graph convolutional network-based methods (GCN [34], SAGE [37]), embedding-based methods (SEAL [21], NBFnet [38]), advanced GNN methods (Neo-GNN [22], BUDDY [23]), and NCN [24], NCNC [24], IECNC. The results highlight the superior performance of our proposed IECNC method

It is noteworthy that traditional methods such as CN, AA, and RA, although straightforward, perform poorly. This might be due to their inability to capture the deeper features within complex network structures. In contrast, graph neural network-based methods significantly improved prediction accuracy, especially notable methods such as SEAL, Neo-GNN, and BUDDY. However, even among these advanced methods, our approach still demonstrated a substantial performance improvement.

These comparative experiments conclusively show that our method can more effectively learn and utilize network structural information, thereby providing more accurate link predictions. These results indicate the potential and practical applicability of our method in handling various network data.

Figure 3 compares the performance of Neo-GNN, BUDDY, NCN, NCNC, and IECNC methods across Cora, Citeseer, and Twitch_PT datasets using the Area Under Curve (AUC) metric for link prediction tasks. On the Cora dataset, IECNC achieves an AUC of 97.70%, outperforming NCNC (95.70%) and NCN (94.25%), demonstrating its effectiveness in academic citation networks. In the Citeseer dataset, IECNC achieves an AUC of 99.23%, surpassing NCNC (97.66%) and NCN (96.37%), highlighting its ability to capture complex relationships in scholarly networks. On the Twitch_PT dataset, IECNC achieves an AUC of 95.26%, compared to NCNC (94.67%) and NCN (93.85%), indicating its robust performance in social network analysis. These findings suggest that the IECNC method, which incorporates information entropy to assess the uncertainty in common neighbor predictions, consistently improves prediction accuracy across various datasets, thereby demonstrating its robustness and effectiveness over traditional methods.

5.3 Ablation Experiment

Table 2 illustrates the impact of different activation functions (sigmoid and softmax) and completion strategies (PC and IEC) on link prediction performance in social networks. sigmoid + PC: Uses the sigmoid function to predict common neighbors between nodes, combined with the Probabilistic Completion (PC) method. sigmoid + IEC: Utilizes the sigmoid function in conjunction with the Information Entropy Completion (IEC) method. softmax + PC: Employs the softmax function to predict common neighbors between nodes and applies the Probabilistic Completion method. softmax + IEC: Uses the softmax function combined with the Information Entropy Completion method. Key insights drawn from the table include: The combinations using the softmax activation function outperform those using the sigmoid activation function across all datasets due to the different efficiency of gradient propagation, with softmax being more suitable for backpropagation.

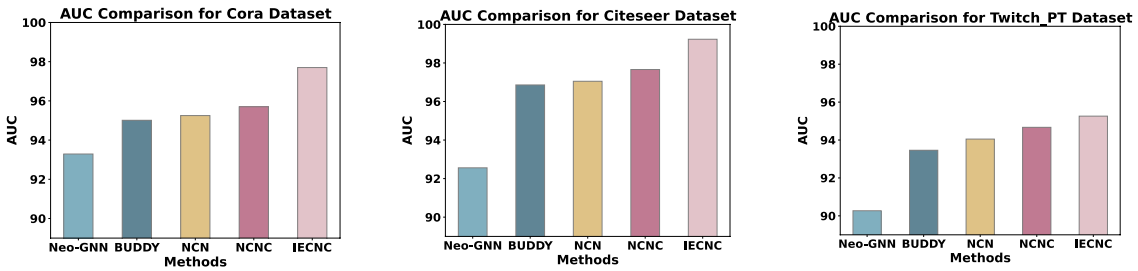


Fig. 3 Comparison of AUC values across different datasets for the NCN, NCNC, and IECNC methods. The AUC values indicate the performance of each method in predicting links within the Cora, Citeseer, and Twitch_PT datasets

Table 2 Ablation study systematically evaluates the impact of various activation functions and completion strategies on link prediction accuracy, as represented by the HITS@100 metric

Method	Cora	Citeseer	Pubmed	twitch_RU	twitch_PT
sigmoid + PC	89.36 ± 1.31	93.99 ± 0.74	81.20 ± 1.54	59.04 ± 1.01	54.34 ± 1.17
sigmoid + IEC	89.96 ± 0.92	93.97 ± 1.11	81.69 ± 1.74	57.07 ± 0.88	53.71 ± 1.21
softmax + PC	93.78 ± 2.88	94.43 ± 1.61	82.54 ± 3.59	58.64 ± 1.35	54.41 ± 1.16
softmax +IEC	**94.68 ± 1.43**	**96.44 ± 1.63**	**87.56 ± 1.51**	**62.18 ± 2.4**	**56.27 ± 1.34**

Bold values indicate the best performance in the experimental results

We analyze the effectiveness of sigmoid and softmax activation functions coupled with Probabilistic Completion (PC) and Information Entropy Completio (IEC) strategies across a range of datasets including Cora, Citeseer, Pubmed, twitch_RU, and twitch_PT

When using the sigmoid activation function, the performance difference between the two completion strategies (PC and IEC) is minimal, indicating a lower sensitivity of sigmoid activation to the completion strategy. When using the softmax activation function, the information entropy completion strategy (IEC) significantly outperforms probabilistic completion (PC), demonstrating that information entropy completion can better utilize and convey the uncertainty information between nodes. The softmax+IEC method shows the best performance across all datasets with a relatively small standard deviation, indicating that our method is not only effective but also stable.

As for Table 3, in this ablation study, we evaluated the performance changes of the IECNC model after removing various core components of P_{uij}, to determine the impact of these components on the overall effectiveness of the model. Each variant removed different parts of the node neighbor completion strategy to test their contributions.

IECNC-$A_{\cap}$: This variant removed the default assumption that nodes connected to both endpoints are their common neighbors. Results show that removing this strategy leads to a slight decrease in performance across most datasets, indicating that direct identification of common neighbors positively affects the model's prediction accuracy.

IECNC-A_{i-j} and IECNC-A_{j-i}: These two variants removed the unilateral neighbor completion strategy, where nodes connected to one endpoint are assumed to have a probability of being common neighbors with the other endpoint. The performance of these variants is generally lower than the complete model but higher than the variant that completely removes the neighbor completion strategy, IECNC-P_{uij}. This suggests that even partial utilization of neighbor information can significantly enhance model performance.

IECNC-P_{uij}: This variant removed the entire neighbor completion strategy. Performance significantly declined across all datasets, demonstrating that the neighbor completion strategy is crucial for enhancing the model's ability to capture and process complex network structures.

Overall, these ablation study results emphasize the importance of the neighbor completion strategy in the IECNC model, especially when dealing with large-scale networks with complex connectivity patterns. The removal of individual components, while leading to a drop in performance, also validates the superiority of IECNC in integrating various network information.

Table 4 aims to validate the effectiveness of entropy in measuring uncertainty by comparing similar probability pairs and their corresponding entropy values. Although the probabilities (P1 and P2) are very close, such as 0.0001401 and 0.0001416, their corresponding entropy values (IE1 and IE2) show subtle differences, at 0.0012433 and 0.0012546, respectively. This indicates that even when probabilities are nearly identical, entropy can still capture potential variations in uncertainty.

The results demonstrate that when probabilities are similar, changes in entropy provide a more accurate quantification of event uncertainty. For example, in the case of P1 = 0.0012376 and P2 = 0.0012393, despite the small difference in probability, the variance in entropy reveals that uncertainty still exists. In contrast, solely relying on probabilities cannot uncover these subtle changes.

Thus, entropy can capture nuances in uncertainty that probabilities alone fail to reveal. Especially when dealing with similar probability events, entropy offers a more refined metric, making it a more effective tool in uncertainty analysis.

Table 4 Comparison of similar probability pairs (P1, P2) and their corresponding entropy values (IE1, IE2)

P1	P2	IE1	IE2
0.0001401	0.0001416	0.0012433	0.0012546
0.0001771	0.0001798	0.0015302	0.0015509
0.0002959	0.0002930	0.0024041	0.0023834
0.0012376	0.0012393	0.0082852	0.0082950
0.0010143	0.0010172	0.0069923	0.0070090
0.0006488	0.0006512	0.0047627	0.0047777

The table demonstrates how entropy captures subtle variations in uncertainty even when the probabilities are nearly identical. This highlights the effectiveness of entropy in quantifying uncertainty compared to solely relying on probabilities

Table 3 Performance comparison of IECNC model variants on multiple datasets

Method	Cora	Citeseer	Pubmed	twitch_RU	twitch_PT
IECNC-$A_{\cap}$	91.08 ± 0.56	93.32 ± 1.74	84.02 ± 0.71	59.71 ± 1.15	53.87 ± 0.91
IECNC-A_{i-j}	92.39 ± 1.97	94.97 ± 1.36	85.78 ± 0.98	60.67 ± 1.17	54.36 ± 1.03
IECNC-A_{j-i}	91.98 ± 1.23	95.02 ± 1.15	85.97 ± 1.05	60.75 ± 1.33	54.23 ± 1.12
IECNC-P_{uij}	90.51 ± 0.97	92.34 ± 1.02	82.17 ± 0.78	58.12 ± 0.96	52.25 ± 1.03
IECNC	**94.68 ± 1.43**	**96.44 ± 1.63**	**87.56 ± 1.51**	**62.18 ± 2.4**	**56.27 ± 1.34**

Bold values indicate the best performance in the experimental results

The results highlight the effectiveness of different components of P_{uij} of the IECNC model, with the integrated IECNC showing the best overall performance(HITS@100)

5.4 Parameter Analysis

In Fig. 4, we analyze the impact of different parameters on Hits@100 values for the Citeseer, Cora, and Twitch datasets:

For batch size variations, Fig. 4a, the Citeseer dataset maintains Hits@100 above 93, showing low sensitivity to batch size changes despite a minor decrease. In the Cora dataset, Hits@100 slightly declines but remains within 93–94. Meanwhile, for Twitch, Hits@100 fluctuates but stays between 53 and 56 with increasing batch size.

Regarding the GNN learning rate, Fig. 4b, Hits@100 for the Citeseer dataset is stable across various rates, demonstrating robustness with consistently high performance. In the Cora dataset, performance is stable yet slightly decreasing, remaining effective overall. For Twitch, Hits@100 shows minor fluctuations but is generally steady with different learning rates.

For the Probscale parameter, Fig. 4c, Citeseer's Hits@100 decreases slightly with increasing Probscale but stays stable, showing low sensitivity. In Cora, Hits@100 declines as Probscale increases, highlighting sensitivity to this parameter. Twitch's Hits@100 varies slightly with Probscale changes but is generally stable.

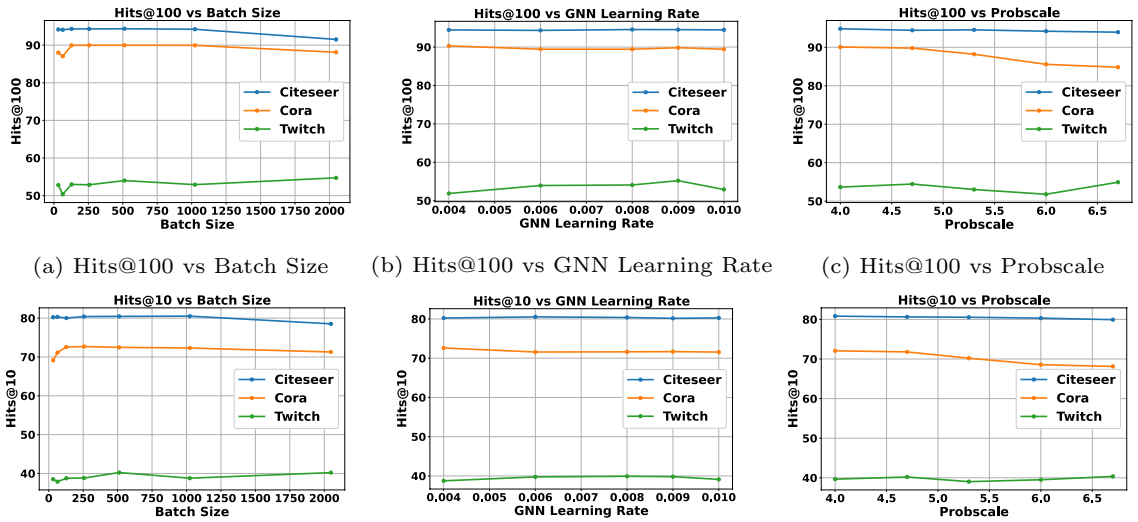
In the Hits@10 metric, Fig. 4d–f, the Cora and Citeseer datasets are insensitive to batch size; however, there is a tendency for performance to decline when the batch size exceeds 1024. On the other hand, performance on the Twitch dataset relatively improves as batch size increases. Cora and Citeseer datasets are also insensitive to changes in GNN learning rate, whereas Twitch achieves relatively higher performance at a GNNLR of 0.008. Citeseer is insensitive to changes in probscale, and an excessive probscale leads to a decrease in performance for the Cora dataset.

Overall, different parameters have varying impacts on the datasets. The Citeseer and Cora datasets show lower sensitivity to parameter changes, and the model's performance is relatively stable on these datasets.

In this study, we investigated the impacts of various parameters on model performance, specifically focusing on batch size, GNN learning rate, and ProbScale. We conducted a thorough evaluation using the Citeseer, Cora, and Twitch datasets, analyzing their Hits@100 and Hits@10 metrics.

Initially, adjustments in batch size demonstrated minimal impact on the Citeseer dataset, with Hits@100 consistently remaining above 93, indicating low sensitivity to changes in batch size. Conversely, the Cora dataset exhibited slight fluctuations in Hits@100, though it generally maintained stability between 93 and 94, underscoring the model's robustness. In contrast, the Twitch dataset showed more pronounced sensitivity, with Hits@100 oscillating between 53 and 56 as batch size increased.

Regarding the GNN learning rate, the Citeseer dataset displayed stable performance across varying rates, reflecting the model's robustness under this parameter. The Cora



(a) Hits@100 vs Batch Size (b) Hits@100 vs GNN Learning Rate (c) Hits@100 vs Probscale

(d) Hits@3 vs Batch Size (e) Hits@3 vs GNN Learning Rate (f) Hits@3 vs Probscale

Fig. 4 Parameter Analysis on Cora, Citeseer, and Twitch datasets. **a** Hits@100 versus Batch Size: The impact of different batch sizes on Hits@100 for Cora, Citeseer, and Twitch datasets. **b** Hits@100 versus GNN Learning Rate: The impact of different GNN learning rates on Hits@100 for Cora, Citeseer, and Twitch datasets. **c** Hits@100 versus Probscale: The impact of different Probscale values on Hits@100 for Cora, Citeseer, and Twitch datasets. **d** Hits@10 versus Batch Size: The impact of different batch sizes on Hits@10 for Cora, Citeseer, and Twitch datasets. **e** Hits@10 versus GNN Learning Rate: The impact of different GNN learning rates on Hits@10 for Cora, Citeseer, and Twitch datasets. **f** Hits@10 versus Probscale: The impact of different Probscale values on Hits@10 for Cora, Citeseer, and Twitch datasets

dataset also showed relative stability, albeit with a minor decline. Meanwhile, the Twitch dataset maintained a general steadiness in Hits@100 across different learning rates, indicating a degree of resilience.

Lastly, adjustments to the ProbScale parameter revealed that while Citeseer's Hits@100 slightly declined with increasing ProbScale, it remained stable overall, demonstrating low sensitivity. The Cora dataset, however, exhibited a higher sensitivity to this parameter, with a notable decrease in Hits@100 as ProbScale increased. Twitch's performance varied slightly with changes in ProbScale but generally stayed stable.Citeseer and Cora datasets exhibited lower sensitivity to parameter changes, maintaining relative stability, whereas the Twitch dataset showed greater sensitivity to certain parameter adjustments.

5.5 Visual Analysis

The t-SNE visualizations, as shown in Fig. 5, offer a comprehensive view of the spatial distribution of entity embeddings across three distinct datasets: Cora, Citeseer, and PubMed, with each dataset's entities differentiated by color-coded labels corresponding to various categories. These visualizations facilitate an insightful exploration into how embeddings capture the semantic relationships between entities within each dataset.

Cora The t-SNE plot for the Cora dataset reveals discernible clustering among entities categorized under topics such as *Genetic Algorithms*, *Neural Networks*, and *Rule Learning*. Despite some overlap, the distinct clusters indicate that the embeddings effectively encapsulate the nuances between different topics, affirming the embeddings' capability to delineate subject matter boundaries within academic literature.

Citeseer In our experiment applying our embedding algorithm to the Citeseer dataset such as *Databases*, *Information Retrieval*, and *Machine Learning* and visualizing it using t-SNE dimensionality reduction, despite some overlap among topics within the dataset, our algorithm successfully differentiated the main categories in two-dimensional space. This demonstrates the algorithm's effectiveness in identifying and distinguishing between related but distinct research areas. These results indicate that even in cases of category intersection and blurred boundaries, our algorithm can learn distinctive feature representations, providing a powerful tool for further academic research and applications.

PubMed we applied our embedding algorithm to the PubMed dataset and visualized the results using t-SNE dimensionality reduction. The results clearly show that our algorithm can significantly distinguish these two main categories in the two-dimensional space. In the plot, the purple points represent 'Diabetes Mellitus, Experimental,' and the yellow points represent 'Diabetes Mellitus Type 1.' It is evident that these two categories form a distinct boundary in the plot, demonstrating the strong capability of the algorithm in capturing and differentiating the features of different categories.

5.6 Training and Evaluation Metrics Comparison

As shown in Fig. 6, in both datasets, the training loss exhibited a rapid decline, indicating that the model could quickly learn useful information from its initial state. During the first few epochs, the loss dropped sharply and then gradually stabilized. This pattern of rapid decline followed by stabilization is typical in the deep learning training process, suggesting that the learning rate and model structure are set appropriately. The Hits@100 were relatively stable across the training, validation, and test sets of both datasets, with little difference between them. This indicates that the model has good generalization ability on the Citeseer dataset and is not significantly affected by overfitting. In summary, the performance of the model on both the Citeseer and Cora datasets reflects its good learning ability and adaptability.

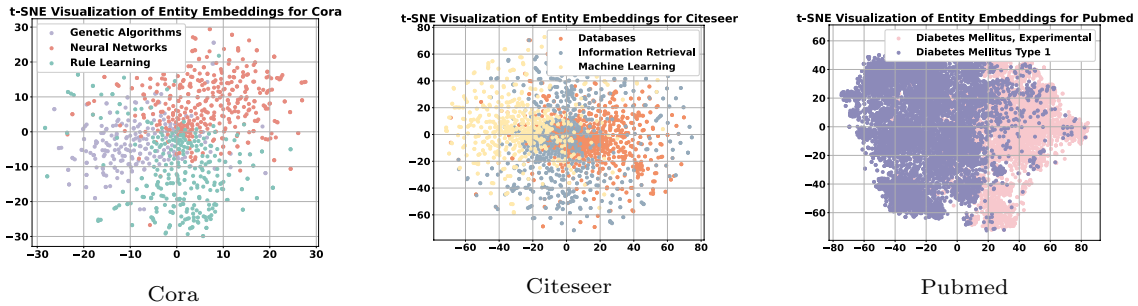


Fig. 5 The t-SNE visualizations depicted here illustrate the entity embeddings for the Cora, Citeseer, and PubMed datasets, demonstrating clear clustering and distinct differentiation among various categories. These visualizations effectively highlight the separation of topics or classes within each dataset, using color-labels to represent different categories, thereby providing a visual affirmation of the embedding's ability to capture and segregate complex data characteristics

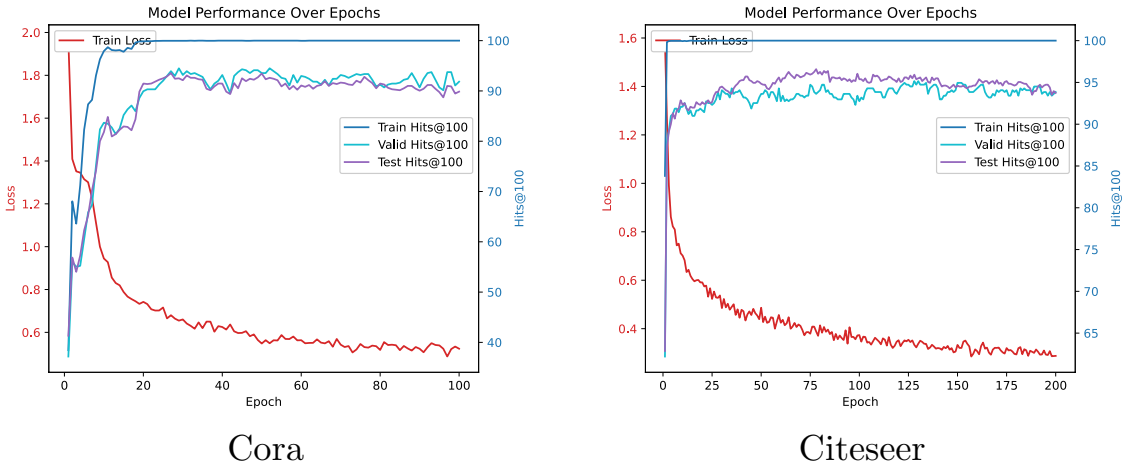


Fig. 6 Training and evaluation metrics comparison on Citeseer and Cora datasets. The figure shows the training loss and Hits@100 metrics across training, validation, and test sets. The rapid decline in training loss and stabilization in Hits@100 indicate that the model quickly learns useful information and has good generalization ability, with minimal overfitting

6 Conclusion

Link prediction plays a crucial role in uncovering hidden relationships within network data, significantly impacting fields such as social network analysis and bioinformatics. This technique is essential for understanding and interpreting complex data structures that traditional analysis methods typically cannot reveal. To address the inherent limitations of traditional link prediction strategies, which often fail to capture deeper relational nuances, we employ the dynamic node functionalities of MPNN. This approach primarily focuses on first-order neighbors and uses set-based aggregation to enhance the relational context between nodes. By first predicting common neighbors and then integrating this information with MPNN, our method not only predicts missing links more accurately but also improves computational efficiency. Additionally, we introduce information entropy into the model to assess the uncertainty of neighboring connections, further optimizing the prediction process and accuracy. Experimental results demonstrate that our method significantly outperforms both traditional and modern link prediction models across various datasets. For instance, on the Cora and Citeseer datasets, our method achieved HITS@100 scores of 94.68% and 96.44%, respectively, which are notably higher than those of other methods.

Acknowledgements This work is partially supported by National Key RD Program of China (2023YFC2705700), the National Natural Science Foundation of China (62206202, 62225113), China Postdoctoral Science Foundation, China (2022M712461), Artificial Intelligence Innovation Project of Wuhan Science and Technology Bureau (No. 2022010702040070).

Authors' contributions Zhengyun Zhou: Conceptualization, Methodology, Formal analysis, Investigation, Writing. Guojia Wan: Validation, Investigation, Supervision, Review, Editing, Funding acquisition. BoDu: Supervision, Review, Editing, Project administration, Funding acquisition.

Funding This work is partially supported by National Key RD Program of China (2023YFC2705700), the National Natural Science Foundation of China (62206202, 62225113), China Postdoctoral Science Foundation, China (2022M712461), Artificial Intelligence Innovation Project of Wuhan Science and Technology Bureau (No. 2022010702040070).

Availability of data and materials The datasets generated and analysed during the current study are available in the Cora, Citeseer, Pubmed and Twitch.

Declarations

Competing interests No potential conflict of interest was reported by the authors.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Trouillon T, Welbl J, Riedel S, Gaussier E, Bouchard G (2016) Complex embeddings for simple link prediction. In: International conference on machine learning. PMLR, pp 2071–2080

2. Mubang F, Hall LO (2022) Vam: an end-to-end simulator for time series regression and temporal link prediction in social media networks. IEEE Trans Comput Soc Syst 10(4):1479–1490
3. Carchiolo V, Cavallo C, Grassia M, Malgeri M, Mangioni G (2022) Link prediction in time varying social networks. Information 13(3):123
4. Stanfield Z, Coşkun M, Koyutürk M (2017) Drug response prediction as a link prediction problem. Sci Rep 7(1):40321
5. Nasiri E, Berahmand K, Rostami M, Dabiri M (2021) A novel link prediction algorithm for protein-protein interaction networks by attributed graph embedding. Comput Biol Med 137:104772
6. Vahidi Farashah M, Etebarian A, Azmi R, Ebrahimzadeh Dastjerdi R (2021) A hybrid recommender system based-on link prediction for movie baskets analysis. J Big Data 8:1–24
7. Afoudi Y, Lazaar M, Hmaidi S (2023) An enhanced recommender system based on heterogeneous graph link prediction. Eng Appl Artif Intell 124:106553
8. Xiao P, Du B, Yun S, Lit X, Zhang Y, Wu J (2018) Probabilistic graph embedding for unsupervised domain adaptation. In: 2018 24th international conference on pattern recognition (ICPR), pp 1283–1288
9. Wang G, Wang Y, Li J, Liu K (2021) A multidimensional network link prediction algorithm and its application for predicting social relationships. J Comput Sci 53:101358
10. Zhang M, Chen Y (2020) Inductive matrix completion based on graph neural networks. In: International conference on learning representations
11. Su Z, Zheng X, Ai J, Shen Y, Zhang X (2020) Link prediction in recommender systems based on vector similarity. Phys A 560:125154
12. Daud NN, Ab Hamid SH, Saadoon M, Sahran F, Anuar NB (2020) Applications of link prediction in social networks: a review. J Netw Comput Appl 166:102716
13. Pech R, Hao D, Lee Y-L, Yuan Y, Zhou T (2019) Link prediction via linear optimization. Phys A 528:121319
14. Poursafaei F, Huang S, Pelrine K, Rabbany R (2022) Towards better evaluation for dynamic link prediction. Adv Neural Inf Process Syst 35:32928–32941
15. Kazemi SM, Poole D (2018) Simple embedding for link prediction in knowledge graphs. Adv Neural Inf Process Syst 31:4289–4300
16. Xiao P, Du B, Wu J, Zhang L, Hu R, Li X (2018) Tlr: transfer latent representation for unsupervised domain adaptation. In: 2018 IEEE international conference on multimedia and Expo (ICME), pp 1–6
17. Xiao P, Qiu P, Ha SM, Bani A, Zhou S, Sotiras A (2023) Sc-vae: sparse coding-based variational autoencoder with learned ista. Available at SSRN 4794775
18. Toprak M, Boldrini C, Passarella A, Conti M (2022) Harnessing the power of ego network layers for link prediction in online social networks. IEEE Trans Comput Soc Syst 10(1):48–60
19. Gilmer J, Schoenholz SS, Riley PF, Vinyals O, Dahl GE (2017) Neural message passing for quantum chemistry. In: International conference on machine learning, pp 1263–1272
20. Kipf TN, Welling M (2016) Variational graph auto-encoders. arXiv preprint arXiv:1611.07308
21. Zhang M, Chen Y (2018) Link prediction based on graph neural networks. Adv Neural Inf Process Syst 31:5171–5181
22. Yun S, Kim S, Lee J, Kang J, Kim HJ (2021) Neo-gnns: neighborhood overlap-aware graph neural networks for link prediction. Adv Neural Inf Process Syst 34:13683–13694
23. Chamberlain B, Shirobokov S, Ross E, Frasca F, Markovich T, Hammerla N, Bronstein M, Hansmire M (2022) Graph neural networks for link prediction with subgraph sketching. In: International conference on learning representations, p 2209
24. X. Wang HY, Zhang M (2024) Neural common neighbor with completion for link prediction. In: International conference on learning representations
25. Grover A, Leskovec J (2016) node2vec: Scalable feature learning for networks. In: ACM SIGKDD International conference on knowledge discovery and data mining, pp 855–864
26. Kumar A, Singh SS, Singh K, Biswas B (2020) Link prediction techniques, applications, and performance: a survey. Phys A 553:124289
27. Yao L, Wang L, Pan L, Yao K (2016) Link prediction based on common-neighbors for dynamic social network. Proc Comput Sci 83:82–89
28. Barabási A-L, Albert R (1999) Emergence of scaling in random networks. Science 286(5439):509–512
29. Newman MEJ (2001) The structure of scientific collaboration networks. Proc Natl Acad Sci 98(2):404–409
30. Liben-Nowell D, Kleinberg J (2007) The link-prediction problem for social networks. J Am Soc Inform Sci Technol 58(7):1019–1031
31. Barabáisi AL, Albert R (1999) Emergence of scaling in random networks. Science 286(5439):509–512
32. Zhou LT, Lv Zhang Y-C (2009) Predicting missing links via local information. Eur Phys J B 71(4):623–630
33. Adamic LA, Adar E (2003) Friends and neighbors on the web. Soc Netw 25(3):211–230
34. Jiang B, Zhang Z, Lin D, Tang J, Luo B (2019) Semi-supervised learning with graph learning-convolutional networks. In: Computer vision and pattern recognition, pp. 11305–11312
35. Wu Z, Pan S, Chen F, Long G, Zhang C, Yu PS (2021) A comprehensive survey on graph neural networks. IEEE Trans Neural Netw Learn Syst 32(1):4–24
36. Lü L, Zhou T (2011) Link prediction in complex networks: a survey. Phys A 390(6):1150–1170
37. Hamilton WL, Ying Z, Leskovec J (2017) Inductive representation learning on large graphs. In: Neural information processing systems
38. Zhu Z, Zhang Z, Xhonneux L-P, Tang J (2021) Neural bellman-ford networks: a general graph neural network framework for link prediction. In: Neural information processing systems